以下为AI生成的文稿笔记的内容

一、图

【考情与地位】

- 定位：**四大基本逻辑结构之四**
- 要求：**基本概念浅显认知**

【核心概念：图】

- 定义：**顶点 + 边** 的组合
- 与树区别：**顶点间可任意连接**，无层次限制
- 空图：**不存在**，**至少含一个顶点**

【边的分类】

- **无向边**：不带箭头
  - 本质：**等价于两条方向相反的有向边**
- **有向边**：带箭头，又称**弧**

【图的分类】

- **无向图**：所有边均为无向边
- **有向图**：所有边均为有向边
  - 无向边可拆分为两条有向边，故含无向边的图仍属**有向图**
  - 考试区分：**给定图要么全有向，要么全无向**

【图的限制规定】

- **无重复边**：两顶点间同方向边仅一条
  - 方向不同则不重复
- **无自环**：顶点**不能指向自身**

【邻接点】

- 定义：**与某顶点直接相连**的顶点
- 无向图：相连即邻接
- 有向图：**方向决定邻接关系**
  - 例：A→B，则B是A的邻接点，但A不是B的邻接点

【子图】

- 定义：原图中**任取部分顶点和边**构成
- 原图本身也是其子图

【带权图】

- 定义：**边附有数值（权值）** 的图

1.图的定义 05:13

【考点：图的最小顶点数】

- 结论：**至少1个顶点**（图不能为空）
- 对比：**树可为空**，图不可为空

【考点：图的度】

- 背景：树是**特殊无向图**
- 分类讨论：**先分有向/无向**

【无向图的度】

- 定义：**某顶点连接的边数**
- 示例：顶点A连3条边，度=3

【有向图的度】

- 分为**出度**与**入度**
- **出度**：该顶点**发出**的边数

- **入度**：该顶点**收到**的边数
- 总度 = **出度 + 入度**

【核心结论（必记）】

- **无向图**：边数 = **总度数 / 2**
  - 原理：**每条边产生2个度**
- **有向图**：边数 = **总入度 = 总出度**
  - 原理：**发出的边必被接收**，入出总量相等

1) 例题:无向图顶点度 08:39

【题目解析】

- 无向图顶点度 = **邻接点个数**
- 易错项B（连通）：**连通≠邻接**
  - 连通可隔点，**邻接必须直接相连**

2) 例题:有向图顶点度 09:14

【题目解析】

- 有向图**先分方向**再讨论
- 顶点v作为**终点**：即**别人射向v**
- 对应概念：**入度**（收到的边数）

3) 例题:有向图顶点度 09:39

【题目解析】

- n个顶点有向图，某顶点度最大：
  - **出度最大**：向其余 **n-1** 个顶点各发一条边
  - **入度最大**：其余 **n-1** 个顶点各发来一条边
- 结论：**最大度 = 2(n-1)**

4) 例题:无向图顶点度 10:44

无向图顶点度

- **已知条件**：无向图，总度数（顶点度数之和）为d
- **结论一应用**：边数 = 总度数 / 2
  - 本题：边数 = **5条**
- **顶点数**：无法确定（多种情况）
- **最大度数求解**：
  - 思路：将**全部5条边**集中于某一顶点
  - 结果：该顶点度数为**5**
  - 极限分析：度数为6需6条边，超出总边数，不可能

有向图顶点度

- **结论二应用**：总出度 = 总入度
  - 出度之和 = d，则入度之和 = **d**
- 本质：发出的边数 = 接收的边数

2.连通图 16:15

【核心结论】

- **无向连通图**：任意顶点间连通，**无顶点落单**
- **最少边数**：n个顶点，最少需 **n-1条边**
  - 推导：以某一顶点为基点，与其余 **n-1个顶点** 各连一条边
  - 必要性：**少一条边**则必有顶点未连通

【补充说明】

- 连通图对边数无上限要求，**多于n-1条边仍为连通图**

1) 例题:连通图顶点数 17:15

例题1：最少边数

- 10个顶点的连通图
- 应用结论：最少边数 = **n-1 = 9条**

例题2：边数之差最大

- 两个无向连通图，各有10个顶点
- 边数最多情况：**完全图**
  - 边数 = 10 × (10-1) / 2 = **45**
- 边数最少情况：**连通图最小边数**
  - 边数 = 10 - 1 = **9**
- 最大差值 = 45 - 9 = **36**

2) 例题:连通图顶点数 18:16

n个顶点无向连通图

- **最少边数**：**n-1**（≥ n-1）
- **最多边数**：**完全图**，边数 = **n × (n-1) / 2**

3.邻接矩阵 **18:37**

【概念定义】

- **邻接矩阵**：图的规范化矩阵表示
- 双向转换：图 ↔ 邻接矩阵

【无向图邻接矩阵构建】

- 矩阵规模：**n × n**（n为顶点数）
- 行列标志：以顶点作为行列标识
- 填充规则：**有边填1，无边填0**
- 对称性：无向边对应**两个对称的1**

【有向图邻接矩阵构建】

- 填充规则：**有向边（a→b）在对应位置填1**
- 无对称性：一条有向边仅对应**一个1**

【带权图邻接矩阵】

- 填充规则：有边处填**权值**
- 无边处填∞（无穷大）

1) 例题:无向图邻接矩阵 21:49

- 无向图邻接矩阵性质：**对称矩阵**
- 原因：每条无向边产生**两个对称的1**

2) 例题:有向图邻接矩阵 22:00

- 邻接矩阵**非对称 ⇒** 图为**有向图**
- 原因：有向边仅产生**一个1**，无对称元素

例题：零的个数（无向图）

- 已知：n个顶点，e条边
- 矩阵总元素数：**n^2**
- 1的个数：每条无向边对应**2个1**，共 **2e**
- 零的个数 = **n^2 - 2e**

3) 例题:邻接矩阵0的个数 23:05

例题：零的个数（有向图）

- 已知：n个顶点，e条边
- 矩阵总元素数：**n^2**
- 1的个数：每条有向边对应**1个1**，共 **e**
- 零的个数 = **n^2 - e**

4) 例题:邻接矩阵示例 23:31

有向图邻接矩阵构建

- 顶点：0、1、2、3、4
- 矩阵规模：**5 × 5**
- 行列标志：以顶点作为行列标识
- 填充规则：**有向边（a→b）在对应位置填1**
- 具体填充：
  - 0→1：填1
  - 0→2：填1
  - 0→4：填1
  - 1→3：填1
  - 2→3：填1
  - 4→2：填1
  - 4→3：填1
- 其余位置：填0

4.邻接表 **24:49**

【核心概念】

- **邻接表**：图的一种链式存储结构，图与邻接表**可相互转换**
- 转换方法：分**有向图**与**无向图**两类

【无向图构建步骤】

- **表头数组**
- 存储所有顶点（如A、B、C）
- 数组下标：**从0开始**（A=0，B=1，C=2）
- **单链表挂接**
- 每个表头后挂接**与该顶点相连的所有顶点**
- 节点存储内容：**邻接顶点的数组下标**（非顶点本身）
- 示例（A-B-C三角形）：
  - A → 1（B）
  - B → 0（A）→ 2（C）
  - C → 1（B）

【有向图构建步骤】

- 表头数组同无向图
- 仅挂接**该顶点指向的出边邻接点**
- 示例（A→B→C）：
  - A → 1（B）
  - B → 2（C）
  - C → 空

1) 例题:邻接表存储结构 26:43

【题目解析】

- 顶点数组：A、B、C、D、E、F（下标0~5）
- 逐顶点分析出边：
  - **A**：连B（1）、D（3）→ 链：1 → 3 → 空
  - **B**：连C（2）、E（4）→ 链：2 → 4 → 空
  - **C**：无出边 → 直接为空
  - **D**：连E（4）→ 链：4 → 空
  - **E**：连C（2）、F（5）→ 链：2 → 5 → 空
  - **F**：连C（2）→ 链：2 → 空

2) 例题:邻接表指针个数 28:10

【考点1：表头数组大小】

- n个顶点 → 数组长度为**n**（选B）

【考点2：无向图指针总数】

- **空指针也是指针**，必须计入
- 计算逻辑：
  - 表头数组：n个顶点 → **n个指针**
  - 每条无向边 = **2条有向边** → 产生**2个指针**
  - 总指针数 = **n + 2e**（选D）

5.深度优先搜索 29:40

【核心概念】

- **搜索 = 遍历**：访问所有节点，**不漏不重**
- **深度优先搜索（DFS）**：一种遍历方案

【遍历规则】

- 起点：指定或**任选一顶点**
- 策略：从起点出发，**任选一个邻接点**深入访问
- 原则：**能走多远走多远**，直到无路可退，再**回溯**至上一分支继续
- 关键特性：**DFS序列不唯一**（邻接点选择顺序不同）

【示例序列】

- 图：A-B-C-D链式结构
- 序列1：A → B → C → D
- 序列2：A → C → B → D（回溯后走另一分支）
- 序列3：A → C → D → B（回溯后走另一分支）

1) 例题:深度优先搜索序列 32:01

【题目分析】

- 图结构：1-2-3-4-5（链式）
- 选项A（从1出发）：1→2→4→5→3，**符合DFS规则**（能走多远走多远）
- 选项B（从1出发）：1→2→5，**错误**（2还有3、4未访问，不应提前走向5）
- 选项C（从2出发）：2→3→1→4，**错误**（1还有5可走，不应走向不直接相连的4）
- 选项D（从2出发）：2→1→3→4，**错误**（1还有5可走，不应走向4）

6.广度优先搜索 33:30

【核心概念】

- **广度优先搜索（BFS）**：与DFS同为图遍历，**策略不同**

【遍历规则】

- 起点：指定或**任选一顶点**
- 策略：**先访问起点所有邻接点**，再依次对每个邻接点执行相同操作
- 邻接点访问顺序**可任意**，但**必须全部访问完**才进入下一层
- 已访问节点**不重复访问**

【示例序列】

- 图：A连接B、C；C连接D
- 序列1：A → B → C → D
- 序列2：A → C → B → D

1) 例题:广度优先搜索序列 35:13

【题目分析】

- 起点：1（四个选项均从1出发）
- 邻接点：2、3、5（**必须最先全部访问**，顺序可调）
- 最后访问：4（唯一剩余节点）
- 结论：**1后紧跟2、3、5，4必在最后** → 选B

2) 例题:广度优先搜索序列 35:46

【题目分析】

- 选项A、B起点：b
  - b的邻接点：a、d、c（**必须最先全部访问**）
  - 最后访问：e
  - 选项B符合（e在最后）→ **选B**
  - 选项A错误：e出现在中间，未满足邻接点优先原则
- 选项C、D起点：a
  - a的邻接点：b、d、c（**必须最先全部访问**）
  - 最后访问：e
  - 选项C、D均错误：e未在最后

7.最小生成树 36:54

【前置概念：生成树】

- 定义：在**无向连通图**中，取**n-1条边**，保持图**连通**且边数最少
- 关键结论：n个顶点连通最少需**n-1条边**
- 特性：**生成树可能不唯一**（取边方案多样）

【最小生成树】

- 定义：给连通图边**赋权值**，所有生成树中**权值之和最小**者
- 示例：4顶点图，三条边权值和分别为7、8、9 → 最小为**7**
- 特性：**最小生成树也可能不唯一**（不同生成树权值和可能相同）

1) 例题:最小生成树定义 39:46

【核心考点】

- 最小生成树定义：**生成树中权值之和最小**的生成树
- 唯一性：**可能不唯一**（可能有一颗或多颗）→ 选C

2) 例题:连通无向图性质 40:06

【考点：无向连通图性质判断】

- **A. 生成树至少还有一个回路**
  - 错误
  - 原因：**生成树无回路**
  - 理解：有回路 ⇒ 可去掉一条边且仍连通 ⇒ 边数非最少 ⇒ 非生成树
- **B. 最小生成树总是唯一的**
  - 错误
  - 原因：**最小生成树可能不唯一**
- **C. 基图的邻接矩阵不一定是对称矩阵**
  - 错误
  - 原因：**无向图邻接矩阵必对称**（一条无向边对应两个1）
- **D. 正确选项**
  - 结论：**生成树边数 = 顶点数 - 1（n-1）**

8.克鲁斯卡尔算法 41:09

【前置概念】

- **生成树**：连通无向图的极小连通子图
- **最小生成树**：生成树中**权值总和最小**者

【核心考点：克鲁斯卡尔算法（Kruskal）】

- 目标：求**无向连通图的最小生成树**
- 操作步骤（三步）：
  - **去边留点**：保留全部顶点，去掉所有边

- **按权值升序选边**：从**权值最小**的边开始选
- **判环取舍**：
  - 选中边后**未形成回路 ⇒ 保留**
  - 选中边后**形成回路 ⇒ 舍弃**

- 终止条件：**已选 n-1 条边** 或 **所有顶点连通**

【易错点】

- 权值相等时：**可任选一条**
- 结果可能不唯一：不同选择可产生**多个等权值最小生成树**

1) 例题:克鲁斯卡尔算法操作 43:54

【例题：克鲁斯卡尔算法实操】

- 初始：去掉所有边，仅保留顶点
- 选边过程（按权值升序）：
  - **权值1**：连接，无圈 ⇒ 保留
  - **权值2**：连接，无圈 ⇒ 保留
  - **权值3**：连接，无圈 ⇒ 保留
  - **权值4**：连接，无圈 ⇒ 保留
  - **权值5**：连接，**形成圈** ⇒ 舍弃
  - **权值6**：连接，**形成圈** ⇒ 舍弃
  - **权值7**：连接，无圈 ⇒ 保留
- 终止：**6个顶点，已选5条边（n-1）** ⇒ 得到最小生成树

9.普里姆算法 45:05

【核心考点：普里姆算法（Prim）】

- 目标：求**最小生成树**（与克鲁斯卡尔算法结果一致）
- 操作步骤：
  - **去边留点**：保留全部顶点，去掉所有边
  - **任选起点**：未指定时任意选择
  - **最小代价吸入新点**：
    - 从当前已选顶点集合出发
    - 选择**权值最小**的边连接一个新顶点
    - 将该顶点吸入集合
  - **重复步骤3**：直至所有顶点被吸入
- 终止条件：**所有顶点均已连通**

【易错点】

- 代价相同：**任选一条边**均可
- 结果可能不唯一：可产生**多个等权值最小生成树**
- 与克鲁斯卡尔算法区别：**Prim按顶点扩张，Kruskal按边选入**

1) 例题:普里姆算法操作 47:21

【例题：普里姆算法实操】

- 初始：去掉所有边，保留顶点
- 起点：任选 **A** 为起点
- 吸入过程（每次选最小代价边）：
  - **代价1**：吸入新点
  - **代价3**：吸入新点
  - **代价4**：吸入新点
  - **代价2**：吸入新点
  - **代价7**：吸入最后一个点
- 结果：与**克鲁斯卡尔算法**所得最小生成树一致

【方法对比】

- **Prim算法**：需反复比较多种吸入方案，过程较繁琐
- **Kruskal算法**：按权值直接选边，操作更快捷
- 建议：**未强制要求时，优先使用克鲁斯卡尔算法**

10.拓扑排序 49:35

【前置概念】

- **有向无环图（DAG）**：
  - 必须为**有向图**
  - **无回路**（无环）
- **拓扑序列**：
  - 对有向无环图顶点排出的序列
  - 条件：若 **A→B 有边**，则序列中 **A 必须在 B 前**

【核心考点：拓扑排序方法】

- 结论：**任意有向无环图必存在拓扑序列**
- 操作步骤（三步）：
  - **选入度为零的顶点**作为序列起点
  - **删除该顶点及其所有出边 - 重复步骤1-2**，直至所有顶点被选出

【易错点】

- 入度为零：**只有出边，无入边**
- 多个入度为零顶点：**任选其一**均可
- 拓扑序列**可能不唯一**

1) 例题:拓扑序列概念 53:01

【例题：拓扑序列概念判断】

- **有向无环图**：一定可以排出**拓扑序列**
- **存在拓扑序列的条件**：必须是**有向无环图**（无回路）
- **存在拓扑序列的图**：一定是**有向无环图**

2) 例题:拓扑序列条件 53:35

【例题：拓扑序列条件选择】

- 正确选项：**D**
- 依据：**有向无环图**才存在拓扑序列
- 操作起点：选**入度为零**的顶点（只有出边，无入边）

3) 例题:拓扑序列操作 53:46

【例题：拓扑序列实操】

- 起点选择：**入度为零**的顶点 ⇒ 仅 **A** 符合
- 排除项：开头非A的选项均错误
- 逐步删除：
  - 选 **A**，删除A及其出边
  - 剩余图中选 **B**（入度为零）
  - 删除B后选 **D**
  - 删除D后选 **C**
  - 最后选 **E**
- 拓扑序列：**A → B → D → C → E**

4) 例题:拓扑序列操作 54:48

【解题过程】

- **起点确定**：仅**顶点3入度为0**，故起点为3
- **第一轮**：删除3后，**仅顶点1入度为0** → 序列：**3,1**
- **第二轮**：删除1后，**仅顶点4入度为0** → 序列：**3,1,4**

- **第三轮**：删除4后，**顶点2、6入度均为0**，任选其一
  - 题目选项选**2** → 序列：**3,1,4,2**
- **第四轮**：删除2后，**仅顶点6入度为0**（顶点5入度不为0） → 序列：**3,1,4,2,6**
- **结论**：选项**D**正确（含5的选项错误）

11.迪杰斯特拉算法 55:46

【问题定义】

- **单源最短路径问题**：确定一个起点，求该点到其他各顶点的**路径权值之和最小**
- **路径值**：路径上**各边权值之和**

【算法定位】

- 解决方法：**迪杰斯特拉算法**
- 核心思想：**逐步确定**起点到各顶点的最短路径

1) 例题:单源最短路径问题 56:09

【算法操作演示】

- **初始**：起点a已确定
- **第一轮**：找离a最近的顶点
  - 候选：b（权值1）、c（权值2）
  - 确定：**b**，路径**a→b**，路径值**1**
- **第二轮**：考虑已确定顶点b的影响
  - a→c：2；a→b→c：4；a→b→d：6
  - 确定：**c**，路径**a→c**，路径值**2**
- **第三轮**：考虑已确定顶点b、c的影响
  - a→b→d：6；a→c→d：6（**任选其一**）
  - 确定：**d**，路径**a→c→d**，路径值**6**

【概念题】

- **迪杰斯特拉算法**：用于求**单源最短路径问题**

2) 例题:求最短路径算法 58:17

【概念辨析】

- **单源最短路径**：起点到其余各顶点**边权值之和最小**的路径
- **采用算法**：**迪杰斯特拉算法**
- **易混淆**：哈夫曼算法求**WPL**；最小生成树算法另有所指
- **最短路径定义**：经过的**边权值之和**最小 → 选**C**

【算法应用：起点a】

- **第一轮**：离a最近的相邻点
  - 候选：c（10）、b（15）、d（14）
  - 确定：**c**，路径**a→c**，路径值**10**
- **第二轮**：考虑c的影响
  - a→b：15；a→c→b：12；a→d：14；a→c→d：13
  - 确定：**b**，路径**a→c→b**，路径值**12**
- **第三轮**：考虑b、c的影响
  - a→d：14；a→c→d：13
  - 确定：**d**，路径**a→c→d**，路径值**13**
- **第四轮**：考虑b、c、d的影响
  - a→c→b→e：19；a→c→e：33；a→c→f：20；a→c→d→f：21
  - 确定：**e**，路径**a→c→b→e**，路径值**19**
- **第五轮**：考虑已确定顶点影响
  - a→c→f：20；a→c→d→f：21
  - 确定：**f**，路径**a→c→f**，路径值**20**

二、直接插入排序 **01:02:03**

【基本概念】

- **排序方向**：升序（从小到大）、降序（从大到小）
- **默认**：升序
- **有序区**：已排好序的部分
- **无序区**：未排好序的部分

【操作原理】

- **核心操作**：每趟将**无序区第一个元素**插入到**有序区正确位置**
- **有序区扩大**：每趟**增加一个元素**
- **排序趟数**：**n-1趟**（第一个元素天然有序，剩余n-1个需插入）

【示例演示：初始序列 5,1,3,2,4】

- **初始**：有序区[5]，无序区[1,3,2,4]
- **第一趟**：插入1 → 有序区[1,5]，无序区[3,2,4]
- **第二趟**：插入3 → 有序区[1,3,5]，无序区[2,4]
- **第三趟**：插入2 → 有序区[1,2,3,5]，无序区[4]
- **第四趟**：插入4 → 有序区[1,2,3,4,5]，无序区空

三、希尔排序 **01:04:50**

1.例题**1:**三趟直接插入排序后的结果

【解题过程】

- **初始序列**：15,10,12,5,48,30
- **第一趟**：插入10 → **10,15**,12,5,48,30
- **第二趟**：插入12 → **10,12,15**,5,48,30
- **第三趟**：插入5 → **5,10,12,15**,48,30
- **三趟后结果**：**5,10,12,15,48,30**

2.例题**2:**应用直接插入排序方法对剩余键值排序的排序过程及稳定性判断 **01:05:53**

【排序过程】

- **初始状态**：有序区[38,45,66,90]，无序区[80,12,15,45]
- **第一趟**：插入80 → 有序区[38,45,66,80,90]
- **第二趟**：插入12 → 有序区[12,38,45,66,80,90]
- **第三趟**：插入15 → 有序区[12,15,38,45,66,80,90]
- **第四趟**：插入45 → 有序区[12,15,38,45,45,66,80,90]
  - **相等元素**：新45放在原45**后面**（稳定性体现）

【稳定性判断】

- **直接插入排序**：**稳定**（相等元素相对位置不变）
- 稳定性概念：**后续集中讲解**

3.知识点**1:**希尔排序的概念及原理 **01:07:40**

【算法定位】

- **本质**：直接插入排序的**改进版**
- **别名**：**缩小增量排序**

【改进动机】

- **长序列效率低**：直接插入排序对超长序列（如3万元素）效率低下
- **改进思路**：先分组排序降低混乱度，再整体排序

【操作原理】

- **分组策略**：按**固定间隔**分组（非连续分段）
- **逐步缩小增量**：

- 例：3万元素 → 分3组（每组1万）→ 组内直接插入排序
- 再分2组（每组 - 5万）→ 组内直接插入排序
- 最后分1组（整体）→ 直接插入排序

- **效率提升原因**：前期分组排序**降低序列混乱度**，使最终整体排序更高效

四、起泡排序 01:09:25

1.希尔排序 01:09:27

【核心思想】

- **增量**：即间隔，将序列按间隔分组
- 对每组分别进行**直接插入排序**
- 逐步缩小增量，直至为1

【示例过程：6元素序列（下标0~5）】

- **增量=3**：
  - 分组：下标（0,3）、（1,4）、（2,5）
  - 各组排序后：**（5,2）→（2,5）**、（1,4）不变、（3,6）不变
- **增量=2**：
  - 分组：下标（0,2,4）、（1,3,5）
  - 排序后：**（2,3,4）**、（1,5,6）不变
- **增量=1**：
  - 对整个序列做**直接插入排序**
  - 最终结果：**1,2,3,4,5,6（升序）**

【关键结论】

- 增量序列可任意设定（如5、10等），但**最后一次增量必须为1**，才能保证序列完全有序

2.例题1:希尔排序的初始增量取值范围 01:11:38

【增量下界】

- 增量最小值为**1**（增量为0无意义，元素位置不变）

【增量上界】

- 下标范围：0 ~ n-1
- 从下标0出发，最大可加至**n-1**（再加则越界）
- 故增量最大值为**n-1**

【结论】

- 增量s取值范围：**1 ≤ s ≤ n-1**

3.例题2:希尔排序的增量序列排序 01:12:23

【题目条件】

- 7个元素：15, 18, 12, 15, 8, 10, 10
- 升序排序，增量序列：**5, 3**

【增量=5】

- 分组：下标（0,5）、（1,6）
- 排序后：**（15,8）→（8,15）**、（18,10）不变
- 其余元素（下标2,3,4）各自成组，不变

【增量=3】

- 分组：下标（0,3,6）、（1,4）、（2,5）
- 排序后：**（8,15,10）→（8,10,15）**、（18,15）不变、（12,8）不变

【最终结果】

- 序列变为：**8, 18, 12, 10, 15, 8, 15**

- 对应选项：**C**

4.起泡排序 01:13:53

【基本操作】

- 从前往后比较**相邻元素**
- 若为**逆序**（前大后小，目标升序）则交换
- 每趟将当前**最大元素**移至末尾正确位置

【排序过程】

- 第一趟：最大元素冒泡至末尾
- 第二趟：次大元素归位（无需再比较末尾已排好元素）
- 依此类推，共需 **n-1趟**

【改进版本】

- 若某一趟排序后**无任何交换**，说明序列已有序
- 后续趟数**可提前终止**

【名称由来】

- 元素像气泡一样逐步上浮至正确位置，故称**起泡/冒泡排序**

5.例题3:冒泡排序的两趟排序结果 01:17:17

【题目条件】

- 7个关键字，冒泡排序升序，求**两趟后结果**

【第一趟】

- 15依次与12、13、8、4、5、1比较，均为逆序，逐一交换
- 结果：**15移至末尾**

【第二趟】

- 对前6个元素重复冒泡
- 12与13正序不动，13与8、4、5、1逆序交换
- 结果：**13移至倒数第二位**

【两趟后序列】

- **12, 8, 4, 5, 1, 13, 15**
- 对应选项：**B**

五、快速排序 **01:18:05**

1.快速排序的基本思想 01:18:08

【核心思想】

- 选取一个元素作为**基准（枢轴）**
- 将**小于基准**的元素放其前，**大于基准**的放其后
- 基准元素即处于**最终正确位置**

【整体有序】

- 基准前后区域内部无需有序，只需保证**整体有序**（前小后大）
- 对前后两个区域**递归**执行相同操作，直至完全有序

【两种划分方法】

- **对调划分**：通过元素对调实现划分
- **转移划分**：通过元素移动填充空位实现划分

2.例题2:对调划分 01:19:44

【操作步骤】

- 选取**第一个元素**为基准（枢轴）
- 将基准与**最后一个元素**对调

- 设置**左右指针**：
  - 左指针从左向右找**大于基准**的元素
  - 右指针从右向左找**小于基准**的元素
- 找到一对即**交换**，继续寻找下一对
- 当左右指针**交叉**时停止，交叉点即基准归位处

【示例：基准42】

- 初始序列：42, 65, 68, 35, 1, 70, 25, 79, 59, 63
- 对调后：63, 65, 68, 35, 1, 70, 25, 79, 59, **42**
- 第一对交换：**65与25** → 25, 68, 35, 1, 70, 65, 79, 59, 63, 42
- 第二对交换：**68与1** → 25, 1, 35, 68, 70, 65, 79, 59, 63, 42
- 指针交叉，基准归位：**42与68交换**
- 最终结果：**25, 1, 35, 42, 70, 65, 79, 59, 63, 68**

【结果特征】

- 基准42前所有元素**小于42**，后所有元素**大于42**

3.例题3:转移划分 01:22:35

【操作步骤】

- 选取**第一个元素**为基准，**空出该位置**
- 右指针从右向左找**小于基准**的元素，填入左侧空位
- 左指针从左向右找**大于基准**的元素，填入右侧空位
- 交替进行，直至左右指针**重合**
- 重合位置填入基准，完成划分

【示例：基准42】

- 初始序列：42, 65, 68, 35, 1, 70, 25, 79, 59, 63
- 右指针找到25（<42），填入首位 → 25, 65, 68, 35, 1, 70, __, 79, 59, 63
- 左指针找到68（>42），填入空位 → 25, __, 68, 35, 1, 70, 68, 79, 59, 63
- 右指针找到1（<42），填入空位 → 25, 1, 68, 35, __, 70, 68, 79, 59, 63
- 左指针移动，35（<42）不动，指针重合
- 重合处填入基准42
- 最终结果：**25, 1, 35, 42, 70, 65, 79, 59, 63, 68**

【关键区别】

- 对调划分与转移划分**最终结果可能不同**，但均保证基准归位

六、简单选择排序 **01:24:49**

【考点定位】

- 快速排序基准概念：**选取元素为枢轴/基准**
- 第一趟排序后基准特性：**位于最终位置，左小右大**
- 例题判定：选项C符合（63>55不应在左，小值不应在右）

【核心操作：简单选择排序】

- 每趟操作：**选出最小元素，与首位交换**
- 第二趟：**选出次小元素，与第二位交换**
- 示例演示（序列5,3,2,4,1）：
  - 第一趟：最小1，与首位5交换 → **1,5,3,2,4**
  - 第二趟：剩余最小2，与5交换 → **1,2,5,3,4**
  - 第三趟：最小3，已在首位 → **1,2,3,5,4**
  - 第四趟：最小4，与5交换 → **1,2,3,4,5**
- 规律总结：**n个元素需n-1趟**，第i趟排出第i小元素，最后剩余自然有序

七、堆排序 **01:27:08**

1.直接选择排序的示例 01:27:10

【例题解析】

- 题目：升序直接选择排序，求第一趟结果
- 操作：**最小元素与首位交换**
- 序列：45,24,39,72,98,45
- 最小元素：24，与首位45交换
- 第一趟结果：**24,45,39,72,98,45**

2.堆排序的概念与操作方法 01:27:30

【堆排序原理】

- 基础：**利用堆的性质**（第五章已介绍建堆、删除堆顶）
- 升序策略：**建立初始大根堆**（前小后大）
- 操作步骤：
  - 将序列建立为**初始大根堆 - 堆顶（最大元素）输出至末尾**（相当于删除堆顶）
  - 剩余元素**重新调整为大根堆 -** 重复输出堆顶，直至排序完成
- 趟数规律：**n个元素需n-1趟**，第i趟选出第i大元素放末尾，最后剩余最小元素无需排序

3.例题1:堆排序的题目演示 01:28:49

【题目条件】

- 未说明升降序时，**默认升序**
- 序列：25,52,64,15,70,33

【第一步：建立初始大根堆】

- 按层序排列为完全二叉树
- 从下往上调整（**上大下小**）：
  - 70与52交换
  - 70与25交换
  - 52与25交换
- 初始大根堆：**70,52,64,15,25,33**

【第一趟排序】

- 堆顶70与末尾33互换 → 70锁死（已排好）
- 剩余元素调整为大根堆：64与33交换
- 第一趟结果：**64,52,33,15,25,70**

【第二趟排序】

- 堆顶64与末尾20互换 → 64锁死
- 剩余调整：20与52交换
- 第二趟结果：**52,20,33,15,64,70**

【第三趟排序】

- 堆顶52与末尾15互换 → 52锁死
- 剩余调整：33与15交换
- 第三趟结果：**33,20,15,52,64,70**

八、归并排序 **01:33:17**

【归并排序定义】

- 特指**二路归并**：两个有序序列合并为一个有序序列
- 示例：1,3 + 2,4 → **1,2,3,4**
- 前提条件：**合并前各序列必须有序**

【操作方法】

- 初始：**每个元素视为独立有序序列**
- 第一轮：两两合并为**2元素有序序列**
- 第二轮：两两合并为**4元素有序序列**
- 依此类推，直至**整个序列有序**

【示例演示】

- 序列：7,3,1,6,5,0,2,9,4,8
- 第一轮合并：3,7 | 1,6 | 0,5 | 2,9 | 4,8
- 第二轮合并：1,3,6,7 | 0,2,5,9 | 4,8
- 第三轮合并：0,1,2,3,5,6,7,9 | 4,8
- 第四轮合并：**0,1,2,3,4,5,6,7,8,9**

【易错点】

- 合并前序列必须有序，如**3,1不可直接合并**

九、基数排序 01:36:06

1.例题1:二路归并排序的排序过程 01:36:07

【例题解析】

- 序列：25,30,21,14,36,3,51
- 第一趟两两合并：25,30 | 21,14 | 36,3 | 51
- 合并后有序化：**25,30 | 14,21 | 3,36 | 51**
- 第二趟前两组合并：14,21,25,30
- 后两组合并：3,36,51
- 第三趟整体合并：**3,14,21,25,30,36,51**

2.基数排序的核心 01:37:01

【核心思想】

- 核心操作：**分配与收集**（发出去再收回来）
- 分配规则：**按个位、十位等位数依次进行**
- 每趟流程：按某一位数分配至0-9桶中，再**按桶顺序依次收回**

3.例题2:基数排序的排序过程 01:37:30

【题目条件】

- 序列均为两位数（一位数视为前补0，如1→01）

【第一趟：按个位分配】

- 分配表（0-9桶）：
  - 个位7：27,97,17
  - 个位1：91
  - 个位0：1
  - 个位3：23
  - 个位4：84
- 收集结果：**91,1,23,84,27,97,17**

【第二趟：按十位分配】

- 分配表（0-9桶）：
  - 十位9：91,97
  - 十位0：1
  - 十位2：23,27
  - 十位8：84
  - 十位1：17
- 收集结果：**1,17,23,27,84,91,97**

【结论】

- 第二趟结果**已为升序排列**，分配-收集策略成功

十、排序的特点 **01:39:52**

【考点定位】

- 章节：第七章第九考点
- 内容：**八种排序特点综合比较**
- 目标：**快速把握每种排序最鲜明特征**

十一、排序的特点简介 **01:40:08**

【八种排序核心特点】

- **直接插入排序**：从**无序区**取首元素插入**有序区**；任意趟次**前面有序区有序**
- **希尔排序**：按**间隔分组**进行插入排序
- **起泡排序**（升序）：每趟将**当前最大**沉底；**末尾若干元素有序**
- **快速排序**：设**基准x**；一趟后**左小右大**；**x落于最终位置**
- **简单选择排序**：每趟将**剩余最小**换至**前端**；**前面元素有序且为全局最小**
- **堆排序**：**不断输出堆顶**；**末尾若干元素为最大**
- **归并排序**：**两两合并**有序子序列
- **基数排序**：**按位分配与收集**（发出收回）

十二、例题:排序特点分析一 **01:41:32**

【题目分析】

- 已知：某序列**第二趟排序结果**（非初始）
- 判断：属于哪种排序

【选项判定】

- **A. 直接插入排序**：☑
  - 两趟后**有序区含3个元素**
  - 观察**前3个元素有序** → 符合
- **B. 冒泡排序**：❌
  - 升序：首趟**最大23应在末尾**（不符）
  - 降序：首趟**最小2应在末尾**（不符）
- **C. 简单选择排序**：❌
  - 升序：首趟**最小2应在首位**（不符）
  - 降序：首趟**最大应在首位**（不符）
- **D. 二路归并排序**：❌
  - 两趟后应为**4元素有序块**
  - 观察块内**乱序** → 不符

【结论】：选 **A**

十三、例题:排序特点分析二 **01:43:31**

【题目分析】

- 已知：某序列**第二趟排序结果**
- 判断：所属排序类型

【选项判定】

- **C. 直接插入排序**：☑
  - 两趟后**前3个元素有序** → 符合
- **简单选择排序**：❌
  - 升序：**最小两个应在最前**（不符）
  - 降序：**最大两个应在最前**（不符）

- **冒泡排序**：❌
  - 升序：**最大两个应在最后**（不符）
  - 降序：**最小两个应在最后**（不符）
- **D. 快速排序**：❌
  - 第一趟后需存在**基准x**满足**左小右大**（或左大右小）
  - 逐一排查：**无任何元素满足** → 不符

【结论】：选 **C**

十四、稳定性简介 **01:45:26**

【稳定性定义】

- **稳定**：排序前后**相同关键字相对次序不变**
  - 例：二与二′，排序后**二′仍在二之后**
- **不稳定**：排序后**相同关键字可能交换次序**
  - 注意：**可能交换，非一定交换**

【八种排序稳定性口诀】

- **稳定**：**插（直接插入）、冒（冒泡）、归（归并）、基（基数）**
- **不稳定**：**选（简单选择）、希（希尔）、快（快速）、堆（堆排序）**

十五、例题:稳定性简介 **01:47:00**

【概念填空】

- 不稳定：两个相同关键字排序前后次序 → **不确定/可能交换**
- 相同关键字次序保持不变 → **稳定**
- 关注相同关键字是否变化的问题 → **算法的稳定性**

十六、例题:稳定排序算法 **01:47:33**

【口诀应用】

- 稳定口诀：**插、冒、归、基**
- 选项分析：
  - 含“冒” → 选 **D**

十七、例题:不稳定排序方法 **01:47:49**

【口诀应用】

- 不稳定口诀：**选、希、快、堆**
- 选项分析：
  - 含“堆” → 选 **A**

十八、排序的时间复杂度 **01:47:59**

1.排序的时间复杂度结论 01:48:03

【考点定位】

- 章节：第七章第11考点
- 来源：**课本290页结论**
- 注意：**以课本为准**（不同书籍略有差异）

【记忆技巧】

- 前三个：**$O(n^2)$**（直接插入、冒泡、简单选择）
- 特殊：**希尔排序 O(n^ - 3)**
- 后三个：**$O(n \log_2 n)$**（快速、归并、堆）
- 特殊：**基数排序 O(d(n+rd))**

2.例题1：排序时间复杂度相关题目 **01:48:44**

【题目1：冒泡最好情况】

- 结论：**O(n)** → 直接定位

【题目2：归并平均情况】

- 结论：**O(n log₂n)** → 选 **C**

【题目3：平均时间复杂度为O(n log₂n)】

- 快速排序：**O(n log₂n)** → 选 **A**
- 冒泡/选择/直接插入：**O(n²)** → 排除

【题目4：最坏情况次数最少】

- 直接插入：O(n²)
- 快速：O(n²)
- 简单选择：O(n²)
- 归并：**O(n log₂n)** → 最小 → 选 **D**

【题目5：最坏情况非O(n²)】

- 直接插入/冒泡/快速：O(n²)
- 堆排序：**O(n log₂n)** → 选 **D**

3.排序的空间复杂度结论 01:50:11

【考点定位】

- 章节：第七章第12考点
- 内容：**空间复杂度结论**

【记忆要点】

- 多数排序：**O(1)**
- 特殊（非O(1)）：**快速排序 O(log₂n)**、**归并排序 O(n)**、**基数排序 O(n+rd)**

4.例题2：排序空间复杂度相关题目 **01:50:38**

【题目1：直接插入排序空间复杂度】

- 非特殊三类 → **O(1)**

【题目2：空间复杂度为O(n)】

- 归并排序：**O(n)** → 选 **D**

【题目3：非O(1)】

- 快速、归并、基数 → 含基数 → 选 **D**

5.查找的基本概念 01:51:04

【章节定位】

- 第八章第一考点
- 主题：**查找（检索）**

【核心概念】

- **查找**：又称**检索**，同义
- **单值查找**：查找**具体值**（如查找3）
- **查找表**：搜索范围
  - 常见结构：**数组、链表、树**
- **四种查找方法**（本章主线）：
  - **顺序查找 - 折半查找 - 树查找**（各种树结构）
  - **哈希方法**

6.例题3：查找表和查找方法 **01:52:23**

【概念补充】

- 四种查找方法：**顺序、折半、树查找、哈希**

- **平均查找长度（ASL）**：
  - 英文缩写：**ASL**
  - 作用：衡量查找方法**效率高低**

7.平均查找长度 01:52:56

【ASL定义】

- 含义：查找目标值**平均所需比较次数**
- 计算：**总查找次数 ÷ 元素个数**
  - 例：4元素，查找次数1+2+3+4=10，ASL=10÷4= - **5**
- 意义：任意选一元素，平均需 - **5次**找到

【后续内容】

- 第八章第二考点：**顺序查找**

8.顺序查找 01:54:04

【核心概念】

- **顺序查找**：以**顺序表**（即**顺序存储的线性表**，本质为**数组**）为查找表
- 操作方式：从**第一个元素**起，依次与**目标值**比对，直至找到或遍历完毕

【查找过程示例】

- 目标值=3，数组[1,2,3,...]：第1次比对1（否）→ 第2次比对2（否）→ 第3次比对3（**找到**）

【关键性质】

- **不要求表内元素有序**：无论有序/无序，只要目标存在，从头到尾遍历必能找到

【核心考点：平均查找长度ASL】

- 结论：**ASL = (n+1)/2**
- 推导：
  - 目标值为第i个元素时，查找次数 = i
  - 查找次数序列：**1, 2, 3, ..., n**（等差数列）
  - 求和：n(n+1)/2，再除以n种可能 → **(n+1)/2**

9.例题4：无序数组的查找 01:56:49

【题目判定】

- 无序数组中查找 → 使用**顺序查找**
- 原因：顺序查找**不依赖有序性**，遍历必能找到

【对比：有序数组的查找策略】

- 有序时**不用顺序查找**：逐个遍历效率低
- 示例：有序1~100，目标60
  - 顺序查找：需从头找到60，**效率低**
  - 改进思路：利用有序性，**先定位区间**（如1~~50、51~~100），确定60在后半段，**跳过前半段**

10.例题5：顺序查找的平均查找长度 01:57:49

【考点应用】

- 直接套用结论：**ASL = (n+1)/2**
- 若遗忘结论，可现场推导（等差数列求和除以n）

11.例题6：有序表的顺序查找 01:57:58

【题目条件】

- 64个元素的有序表，顺序查找

【最坏情况分析】

- 情况1：目标值为最后一个元素（如64）→ 需比较 **64次**
- 情况2：目标值不在表中（如100）→ 遍历全部64个元素后确认未找到 → 需比较 **64次**

【结论】

- 最坏情况下比较次数 = **n次**（即64次）

12.折半查找 01:58:34

【基本概念】

- **折半查找**（又称**二分查找**）：对顺序查找的改进，大幅提升效率
- **使用前提（两个必要条件）**：
  - **顺序表**（数组）
  - **元素有序**

【查找思路】

- 取**中间元素**与目标值比对
  - 相等 → **查找成功**
  - 目标值 < 中间值 → 目标在**前半段**，舍弃后半段
  - 目标值 > 中间值 → 目标在**后半段**，舍弃前半段
- 精髓：每次比较**缩小一半查找范围**

【示例】

- 有序数组[- .7]，目标值3：
  - 中间值4（>3）→ 范围缩至[1,2,3]
  - 中间值2（<3）→ 范围缩至[3]
  - 找到3，共3次比较

【偶数个元素处理】

- 无正中间元素时，取**前半段的最后一个元素**作为中间值

【核心考点：折半查找判定树】

- 定义：将折半查找过程表示为**二叉树**
- 构造规则：每次的**中间元素**作为根/子树根，左右子表分别构造左右子树
- 示例（1~8）：根为4，左子树根为2，右子树根为6，依此类推

【判定树性质】

- 每个元素的**查找次数 = 所在层数 + 1**（根节点在第0层）
- 用途：
  - 模拟查找过程（路径即比较序列）
  - **求ASL**：各元素查找次数之和 ÷ 元素个数

13.例题7：能用二分查找的数据结构 02:03:54

【题目判定】

- 二分查找（折半查找）前提：**有序的顺序表**
- 顺序表 = **顺序存储** → 排除链式存储选项
- 必须**有序** → 排除无序数组选项
- 正确选项：**有序的顺序存储结构**

14.例题8：二分查找的过程模拟 02:04:19

【题目1：查找关键字9】

- 10个元素（偶数）→ 中间值取**第5个**（前半段最后一个）
- 比较序列：
  - **32**（9<32）→ 范围缩至前5个
  - **3**（9>3）→ 范围缩至[9,12]
  - **9**（相等）→ **找到**
- 答案：比较序列为 **32, 3, 9**

【题目2：查找关键字32】

- 8个元素（偶数）→ 中间值取**第4个**
- 比较序列：
  - **18**（32>18）→ 范围缩至后4个
  - **32**（相等）→ **找到**
- 答案：依次比较的键值为 **18, 32**

15.例题9：二分查找判定树的构造 02:05:51

【构造过程（8个元素）】

- 前4后4 → 根节点：**25**
- 左子树（前3个元素）：中间值 **13** 为根，左右子节点 **2**、**18**
- 右子树（后4个元素）：中间值取前半段最后一个 → **33** 为根
  - 左子节点：**30**
  - 右子树（2个元素）：中间值 **36**，右子节点 **55**

【判定树结构】

- 根：25
  - 左：13（左：2，右：18）
  - 右：33（左：30，右：36（右：55））

16.例题10：二分查找的平均查找长度 02:06:33

【题目条件】

- 13个关键字的有序表，二分查找，求ASL

【判定树构造（设元素1~13）】

- 根：**7**（前6后6）
- 左子树（1~6）：中间值 **3**（左：1（右：2），右：5（左：4，右：6））
- 右子树（8~13）：中间值 **10**（左：8（右：9），右：12（左：11，右：13））

【ASL计算】

- 各层查找次数：
  - 第0层（1个）：1次 → 1×1=1
  - 第1层（2个）：2次 → 2×2=4
  - 第2层（4个）：3次 → 4×3=12
  - 第3层（6个）：4次 → 6×4=24
- 总和：1+4+12+24 = **41**
- ASL = 41 ÷ 13 = **41/13**

17.索引顺序查找 02:08:38

【考情与地位】

- 定位：**第八章第四考点**，基本不考，了解思想即可
- 别名：**分块查找**（同一概念）

【核心概念：分块有序】

- 前提条件：**分块有序**
  - 块内：**无需有序**
  - 块间：**必须有序**（前一块最大值 < 后一块最小值）

【查找流程】- 建立索引表：从每块中选取**最大值**作为索引（即目录）

- **索引表特性**：天然构成**有序顺序表 - 索引查找**：对索引表采用**折半查找**，快速定位目标所在块
- **块内查找**：块内无序，采用**顺序查找**

【示例演示】

- 三块数据：块1最大8，块2最大17，块3最大29

- 目标值23：因23>17，直接定位第三块，块内顺序查找命中

18.例题11：分块查找的概念 02:10:53

【考点速记】

- **分块查找 = 索引顺序查找**（同一概念）
- 考查形式：概念辨析，了解别名即可

19.二叉查找树 02:11:04

【考情与地位】

- 定位：**第八章第五考点**，新概念
- 别名：**二叉排序树 / 二叉搜索树 / 二叉查找树**（同一概念）

【定义与规则】

- 前提：首先必须是**二叉树**
- 核心规则：**左小右大**
  - 对**每一个节点**：其**左子树所有节点值 < 该节点值 < 右子树所有节点值**

【判定要点】

- 必须满足**所有节点**的左右子树关系
- 反例：若某节点左子树中存在大于该节点的值，则**违反规则**，非二叉查找树

【查找过程】 - 从**根节点**开始，与目标值比较

- 目标值 < 根节点值 → 进入**左子树 -** 目标值 > 根节点值 → 进入**右子树 -** 循环上述过程直至找到或为空

【示例演示】

- 目标值17：根节点10 → 17>10入右子树 → 节点15 → 17>15入右子树 → 找到17
- 目标值8：根节点10 → 8<10入左子树 → 节点7 → 8>7入右子树 → 找到8

20.例题12：二叉查找树的查找过程 02:13:00

【查找流程演示】

- **目标值17**：根节点10 → 17>10 → 右子树 → 节点15 → 17>15 → 右子树 → **找到17**
- **目标值8**：根节点10 → 8<10 → 左子树 → 节点7 → 8>7 → 右子树 → **找到8**

【核心要点】

- 每次比较后**排除一半子树**，缩小查找范围
- 查找路径完全由**节点值大小关系**决定

21.二叉查找树的生成 02:13:49

【生成机制】

- 本质：**不断插入元素**的过程
- 关键结论：**相同元素，插入顺序不同 → 生成的二叉查找树不同**

【插入规则】

- 每个新元素**必须从根节点开始比较**
- 按**左小右大**原则逐层下降，直至找到空位插入

【易错警示】

- **禁止**跳过根节点直接与子树节点比较
- 必须沿完整路径从根节点逐层比较

22.例题13：二叉排序树的插入过程 02:14:06

【示例：相同元素不同顺序】

- 元素集合：{1,2,3,4,5}

顺序一：1,2,3,4,5

- 1为根 → 2在右 → 3在右 → 4在右 → 5在右

- 结果：**单支右斜树**

顺序二：5,4,3,2,1

- 5为根 → 4在左 → 3在左 → 2在左 → 1在左
- 结果：**单支左斜树**

顺序三：3,5,1,4,2

- 3为根 → 5在右 → 1在左 → 4在5左 → 2在1右
- 结果：**较平衡树**

【核心结论】

- **插入顺序决定树形结构**
- 每次插入必须**从根节点开始**比较

【重要性质】

- **中序遍历二叉排序树 → 递增序列（升序）**
- 原理：左小右大 + 中序（左-根-右）→ 天然升序

23.例题14：二叉排序树的性质 02:16:38

【题目解析】

- 条件：节点n是节点m的**孩子**
- 分析：
  - n为m的**左孩子** → n < m
  - n为m的**右孩子** → n > m
- 结论：**大小关系不确定**（取决于左右孩子）

【考点】

- 二叉排序树中，父子节点大小关系**取决于孩子方位**

24.例题15：二叉排序树的中序遍历 02:17:05

【核心结论】

- **中序遍历二叉查找树 → 升序序列**
- 示例：中序遍历结果 **1,3,5**（递增）

【原理】

- 中序顺序：**左子树 → 根 → 右子树**
- 结合左小右大规则 → 天然升序

25.例题16：高度最低的二叉排序树 02:17:23

【题目要求】

- 给定四组插入顺序，构造二叉排序树
- 求**高度最低**（层数最少）者

【选项分析】

- **A**：9,7,2,1,4,10 → 高度较高（左支较长）
- **B**：6,4,1,8,10,5 → **高度最低（最平衡）**
- **C**：5,1,2,6,3,4 → 高度较高
- **D**：2,4,7,5,8,10 → 高度较高

【结论】

- 答案：**B**
- 关键：插入顺序影响树形，**较平衡的插入顺序 → 高度更低**

26.例题17：二叉排序树的构造 02:19:15

【题目条件】

- 9个节点，值1~9，填入给定二叉树结构，满足二叉查找树

【推理过程】 - 根节点A：左右各4个节点 → A为中间值 → **A=5 - 左子树**：值1~4

- **B**：剩余3个全在右 → B为最小 → **B=1**
- **D**：剩余2个全在左 → D为最大 → **D=4**
- **H>F → H=3，F=2 - 右子树**：值6~9
- **C**：剩余3个全在左 → C为最大 → **C=9**
- **E**：剩余2个全在右 → E为最小 → **E=6**
- **I>G → I=8，G=7**

【方法总结】
- 利用**左右子树节点数量**确定根节点值
- 递归应用**左小右大**规则逐层推理

27.二叉查找树的删除 **02:20:46**

【考情与地位】
- 定位：**第八章第六考点**
- 核心要求：删除后**保持左小右大规则**

【删除三种情况】

情况一：叶节点
- 操作：**直接删除**
- 示例：删除9或14，直接移除

情况二：只有一棵子树
- 操作：**用子树直接取代**被删节点
- 示例：删除8（仅右子树）→ 9直接上位；删除15（仅左子树）→ 14直接上位

情况三：左右子树均存在
- 操作：用**前驱或后继**取代被删节点
  - **前驱**：**刚好小于**被删节点的值（非任意左子树节点）
  - **后继**：**刚好大于**被删节点的值
- 示例：删除根节点10
  - 前驱法：**9**取代10
  - 后继法：**14**取代10

【易错点】
- 前驱 ≠ 左子树最大值（需为**直接前驱**）
- 后继 ≠ 右子树最小值（需为**直接后继**）

十九、二叉排序树的操作 **02:22:54**

1.例题**:**二叉树操作判断 **02:22:55**

【考点：操作对高度的影响】
- **删除关键字**：高度**不一定减一**（如删除后结构不变）；**不一定不变**（如删除后高度减一）
- **插入节点**：高度**不一定加一**（如插入后结构不变）
- **查找操作**：**不增删元素**，高度**必然不变**（正确项）

【考点：二叉排序树插入与删除】
- **插入新节点F**：按字母序比较，F > C > D，插入位置为**D右下方**
- **删除节点C**（左右子树均存在）：
  - 规则：用**前驱或后继**替代
  - 前驱法：**B**（刚好小于C）替代C
  - 后继法：**D**（刚好大于C）替代D，F上移接续

【考点：删除节点22】
- 22左右子树均存在，用**前驱20**替代
- 18处理：视为**新插入节点**，按序比较（18 > 17），置于**17右侧**

- 或用**后继24**替代，24原位置删除

二十、**AVL树 02:25:06**

【考情与定位】

- 考点：**AVL树**（第八章第七考点）
- 考频：**较低**，可暂放；若考再回头研究

【概念：AVL树】

- 名称来源：**发明者名字缩写**
- 目的：解决**二叉查找树不均衡**问题（同元素可构造不同形态树）
- 理想形态：**树尽量矮、均衡**（链状极不均衡，满排状最优）

【核心概念：平衡因子】

- 定义：**节点左子树高度 - 右子树高度**
- 合法取值：**-1、0、1**
- 失衡判定：**超出上述三值**即失衡
- 示例：
  - 节点3：左高2，右高0 → 平衡因子**2**（失衡）
  - 节点2：左高1，右高0 → 平衡因子**1**
  - 节点1：左右高0 → 平衡因子**0**

【判定标准】

- **AVL树**：所有节点平衡因子 ∈ **{-1, 0, 1}**
- 非AVL树：存在节点平衡因子**超出合法范围**

1.例题:插入调整AVL树 02:27:43

【考点：AVL树插入调整流程】

- 插入过程与**二叉查找树相同**，每步检查**平衡因子**
- 失衡时进行**旋转调整**（左旋/右旋）

【逐步演示：插入1-7】

- **插入1**：根节点，平衡
- **插入2**：2 > 1，置于右侧，平衡因子-1，未失衡
- **插入3**：3 > 2，置于右侧，节点1平衡因子**-2**（失衡）
  - 调整：对节点1**左旋**，3上移，1下移，恢复平衡
- **插入4**：4 > 3，置于右侧，未失衡
- **插入5**：5 > 4，置于右侧，节点2平衡因子**-2**、节点3平衡因子**-2**（双失衡）
  - 调整：调整**离插入节点最近**的失衡节点3，对3**左旋**，恢复平衡
- **插入6**：6 > 5，置于右侧，节点2平衡因子**-2**（失衡）
  - 调整：对节点2**左旋**，2下移，3上移；**冲突处理**：3作为2的**右孩子**，1仍为左孩子
- **插入7**：7 > 6，置于右侧，节点5平衡因子**-2**（失衡）
  - 调整：对节点5**左旋**，5下移，6上移，恢复平衡

【最终结果与题目要求】

- 最终树：所有节点平衡因子均为**0**
- 题目问：**平衡因子为0的分支节点**（叶节点不计）
- 分支节点：**前3个**（根及中间层节点）
- 答案：**D**

【备注】

- AVL调整过程**较复杂**，考频低，可**暂放**

二十一、**B树 02:32:28**

【背景与定位】

- 定位：**二叉查找树的改进**（较AVL树改进更彻底）

- 核心：针对二叉查找树缺点进行结构性优化

【二叉查找树的缺点】

- **分支少**：最多仅两个分支
- **可能不平衡**：如单支树，左右失衡
- **节点容量小**：每节点仅存一个关键字

【B树的改进措施】

- **多路分支**：支持多叉（如三阶、四阶），提升效率
- **完全平衡**：所有叶结点位于同一层
- **节点多关键字**：每节点可存多个关键字
- **保持有序性**：节点内关键字升序，节点外左小右大

【m阶B树定义（核心考点）】

- **每个节点最多m个子树**（m阶即m叉）
- **所有叶结点在同一层**：维持平衡性
- **节点内关键字升序**，节点外左小右大
- **非空根结点**：
  - 子树数：**最少2个，最多m个**
  - 关键字数：**最少1个，最多m-1个**
- **非根结点**：
  - 子树数：**最少⌈m/2⌉个，最多m个**
  - 关键字数：**最少⌈m/2⌉-1个，最多m-1个**
- 规律：**关键字个数 = 子树数 - 1**

1.例题:B树定义判断 02:39:39

【题目要求】：选出**不符合**B树定义的一项

【选项分析】

- **A**：每个节点最多m个子树 → **符合**（m阶即m叉）
- **B**：所有叶结点在同一层 → **符合**（平衡性要求）
- **C**：非空根结点最少2个子树，最多m个 → **符合**（根结点规定）
- **D**：每个节点关键字最多m个 → **不符合**（应为**m-1个**）

【答案】：**D**

2.例题:B树叙述判断 02:40:34

【题目要求】：选出**正确**的一项

【选项分析】

- **A**：终端节点（叶结点）可位于不同层 → **错误**（所有叶结点**同一层**）
- **B**：非终端节点最多m+1个子树 → **错误**（最多**m个**）
- **C**：非空根结点最少2个关键字 → **错误**（最少**1个**）
- **D**：非根结点关键字范围：⌈m/2⌉-1 至 m-1 → **正确**（直接结论）

【答案】：**D**

3.例题:B树关键字个数 02:41:33

【考点】：非根结点关键字个数范围

【公式】：**⌈m/2⌉ - 1**（下限），**m - 1**（上限）

【计算示例】

- **m=3**：⌈3/2⌉=2，2-1=**1**
- **m=25**：⌈25/2⌉=13，13-1=**12**

4.例题:B树关键字个数 02:42:00

【题目条件】

- **高度为2**的四阶B树（m=4）
- 求**最多**保存的关键字数

【解题思路】

- 使每个节点分支数、关键字数均取**上限**
- 根结点：最多**4个子树**，关键字**3个**（m-1）
- 每个叶结点：最多**3个关键字**（m-1）
- 叶结点数：**4个**

【计算】

- 总关键字数 = 根结点3个 + 4×3个 = **15个**

5.B树的插入 02:42:54

【插入规则】

- 与二叉查找树类似：按**左小右大**规则逐元素插入
- 需维持：**有序性**、**分支数限制**、**关键字数限制**

【核心步骤：分裂】

- 触发条件：节点关键字数**超过上限**（m-1）
- 操作：**中间关键字上提**，左右分裂为两个节点
- 若上提后父节点仍超限，则**继续上提分裂**（逐层进行）

【示例：三阶B树插入3】

- 根结点已有1、2，插入3后超限（上限2）
- 分裂：**2上提**，1与3分列两侧

1) 例题:B树插入操作 02:44:56

【题目条件】

- **三阶B树**（m=3，关键字上限2）
- 插入**65**

【插入路径】

- 65 > 53 且 < 90 → 走**中间分支**
- 插入后该节点含61、65、71 → **超限**（3>2）

【分裂过程】

- **第一轮分裂**：65上提，61与71分列
- 根结点变为53、65、90 → **再次超限**
- **第二轮分裂**：65继续上提，53与90分列
- 最终根结点关键字：**65**

【答案】：D（65）

2) 例题:B树插入过程 02:46:28

【题目条件】

- **三阶B树**：每节点关键字数 **≤2**，达 **3个即分裂**

【插入过程分步】 - 插入35：根节点 → **35 - 插入16**：比35小 → 置于左侧 → **16, 35 - 插入18**：有序插入 → **16, 18, 35** → **达3个，触发分裂**

- 分裂结果：**18上提为根**，左子树 **16**，右子树 **35 - 插入20**：比18大、比35小 → 右子树 → **20, 35 - 插入5**：比18小 → 左子树 → **5, 16 - 插入50**：比18大 → 右子树 → **20, 35, 50** → **超容量，触发分裂**
- 分裂结果：**35上提**至根节点 → 根为 **18, 35**
- 右子树分裂为 **20** 与 **50**（三阶可容纳 **3支**）
- **插入22**：位于18与35之间，且比20大 → 右子树 → **20, 22 - 插入60**：比35大 → 右子树 → **50, 60 - 插入3（关键步骤：连锁分裂）**
  - 比18小、比5小 → 左子树 → **3, 5, 16** → **超容量，触发分裂**
  - **5上提**至根节点 → 根变为 **5, 18, 35** → **再次超容量，触发连锁分裂**

- **18上提**为新根 → 最终根为 **18**
- 分裂后结构：
  - 左子树根 **5**，其左支 **3, 16**，右支 **22**
  - 右子树根 **35**，其左支 **20**，右支 **50, 60 - 插入17**：比18小、比5大 → 中间子树 → 比16大 → **16, 17 - 插入45**：比18大、比35大 → 右子树 → **35, 50, 60** → **超容量，触发分裂**
- **50上提**至根节点 → 根为 **18, 50**
- 右子树分裂为 **35** 与 **45, 60 - 插入7**：比18小、比5大 → 中间子树 → **5, 16, 17** → **超容量，触发分裂**
- **16上提**至根节点 → 根为 **16, 18, 50** → **再次超容量，触发连锁分裂**
- **18上提**为新根 → 最终根为 **18**
- 分裂后结构：
  - 左子树根 **16**，其左支 **5, 7**，右支 **17**
  - 右子树根 **50**，其左支 **35, 45**，右支 **60**

【核心结论】

- **连锁分裂**：下层分裂导致上层超容量时，需**继续向上分裂**
- 三阶B树分裂后**必须保持3支**，不可出现4支

二十二、哈希方法 02:50:00

【定义与别名】

- **哈希方法 = 散列方法**（同义不同叫法）
- 核心目标：将关键字存入数组，确定**每个关键字的存储地址**

【核心机制】

- **哈希函数（散列函数）**：关键字与存储地址间的映射关系
  - 记作 **h(key)**，h取自哈希英文首字母
  - 例：h(50)=3 → 关键字50存于地址3
- **哈希表（散列表）**：存放关键字的数组

【哈希函数构造：除留余数法】

- 表达式：**h(key) = key mod p**（对某数取余）
- 取余含义：**去掉整除部分，取剩余余数**
  - 例：8 mod 7 = 1（7整除7，余1）

【核心概念：冲突】

- **冲突**：两个**不同关键字**经哈希函数算出**相同地址**
  - 例：8 mod 7 = 1，15 mod 7 = 1 → 地址相同，发生冲突
- **同义词**：发生冲突的两个关键字互称同义词

【后续衔接】

- 冲突解决方法：后续考点详述

1.例题:哈希方法概念题 02:53:13

【概念填空1】

- 关键字到存储地址的**映射**（即函数）→ **哈希函数 / 散列函数**

【概念填空2】

- 两个不同关键字经哈希函数算出**地址相同** → 发生**冲突**
- 冲突的两个关键字互称**同义词**

【除留余数法：除数选择原则】

- 经验法则：取**比数组容量略小的素数（质数）**
- 素数定义：**只能被1和自身整除**的数

2.例题:散列函数除数选择 02:54:07

【解题要点】

- 除数选择：**素数（质数）**

【选项分析】

- 表长 **40** → 选**比40略小的质数**
- **39**：可被3整除 → 非质数
- **38**：可被2整除 → 非质数
- **37**：**质数** → 正确选项

3.例题:散列存储计算 02:54:23

【题目条件】

- 散列函数：**除9取余**（虽非质数，但题目允许）
- 任务：计算各关键字地址，统计**地址为1的个数**

【逐项计算】

- **7** mod 9 = **7**
- **34** mod 9 = **7**（27÷9=3，余7）
- **54** mod 9 = **0**（54÷9=6，余0）
- **25** mod 9 = **7**（18÷9=2，余7）
- **64** mod 9 = **1**（63÷9=7，余1）
- **50** mod 9 = **5**（45÷9=5，余5）
- **19** mod 9 = **1**（18÷9=2，余1）

【结果统计】

- 地址为1的关键字：**64、19** → 共 **2个**
- 地址为7的有3个（7、34、25），存在**冲突**，但题目未要求解决

二十三、线性探测法 02:55:20

【定义与别名】

- **线性探测法** = **线性探查法**（同义不同叫法）
- 用途：解决哈希表（散列表）中的**地址冲突**

【核心规则】

- 冲突时：**向后挪一位**，探测下一地址是否为空
  - 空 → 放入
  - 仍冲突 → **继续后挪一位**，直至找到空位

【示例演示】

- 设 h(50)=1，但地址1已被60占据
  - 第1次探测：地址1 → 已占
  - 后挪至地址2 → 空 → **50放入地址2**
- 若地址2也被70占据
  - 继续后挪至地址3 → 空 → **50放入地址3**

1.例题:线性探测法计算 02:56:38

【题目条件】

- 散列表表长 **7** → 地址范围 **0~6**
- 散列函数：**对7取余**
- 冲突处理：**线性探测法**
- 已有关键字：21（地址0）、8（地址1）、25（地址4）

【计算过程】

- 待插入：**14**
- 14 mod 7 = **0** → 应放地址0

- 地址0已占（21） → **后挪探测地址1**
- 地址1已占（8） → **再后挪探测地址2**
- 地址2为空 → **14放入地址2**

【答案】

- 14的下标为 **2**

2.例题:线性探测法计算 02:57:33

【题目条件】

- 散列表长度 **11** → 地址范围 **0~10**
- 散列函数：**对11取余**
- 冲突处理：**线性探测法**
- 已有关键字：15（地址4）、27（地址5）、39（地址6）

【计算过程】

- 待插入：**60**
- 60 mod 11 = **5**（55÷11=5，余5） → 应放地址5
- 地址5已占（27） → **后挪探测地址6**
- 地址6已占（39） → **再后挪探测地址7**
- 地址7为空 → **60放入地址7**

【答案】

- 60存放位置为 **7**

3.例题:线性探测法探查次数 02:58:10

【题目条件】

- 散列表长度 **14** → 地址范围 **0~13**
- 散列函数：**对13取余**
- 冲突处理：**线性探测法**
- 已有关键字：16（地址3）、30（地址4）、18（地址5）

【计算过程】

- 待插入：**29**
- 29 mod 13 = **3**（26÷13=2，余3） → 应放地址3
- **第1次探测**：地址3 → 已占（16）
- **第2次探测**：地址4 → 已占（30）
- **第3次探测**：地址5 → 已占（18）
- **第4次探测**：地址6 → **空** → **29放入地址6**

【答案】

- 探测次数为 **4次**（选D）

二十四、二次探测法 **02:59:08**

【定义与别名】

- **二次探测法** = **二次探查法**（同义不同叫法）
- 用途：解决哈希表冲突的另一种方法

【核心规则】

- 冲突时**重新计算地址**，公式为：
  - **$d \pm 1^2, d \pm 2^2, d \pm 3^2, \ldots$**（依次交替加减）
- 探测顺序：**$d+1^2 \rightarrow d-1^2 \rightarrow d+2^2 \rightarrow d-2^2 \rightarrow d+3^2 \rightarrow d-3^2 \ldots$**

【示例演示】

- 设 h(50)=1，地址1已占
  - 计算 **$1+1^2=2$** → 地址2空 → **50放入地址2**
- 若地址2也占

- 计算 **1-1²=0** → 地址0空 → **50放入地址0**

【与线性探测法区别】

- 线性探测法：**依次后挪一位**
- 二次探测法：**按平方规律跳跃探测**

1.例题:二次探测法计算 03:00:31

【题目条件】

- 哈希表长 **14** → 地址范围 **0~13**
- 哈希函数：**对11取余**
- 冲突处理：**二次探测法**
- 已有关键字：15（地址4）、27（地址5）、39（地址6）、50（地址7）

【计算过程】

- 待插入：**49**
- 49 mod 11 = **5**（44÷11=4，余5） → 应放地址5
- 地址5已占（27） → **二次探测**
  - **第1次**：5+1²=**6** → 已占（39）
  - **第2次**：5-1²=**4** → 已占（15）
  - **第3次**：5+2²=**9** → **空** → **49放入地址9**

【答案】

- 49存放位置为 **9**（选D）

二十五、链地址法 **03:01:50**

【定义与别名】

- **链地址法**（又称**拉链法**）
- 处理冲突策略：**不另寻地址**，将**所有冲突关键字**存于**同一地址**，以**单链表**链接

【处理流程】

- 散列表地址：0~4（共5个位置）
- 示例：
  - 关键字5 → 散列地址1 → 挂接于地址1链表
  - 关键字10 → 散列地址1 → 冲突 → **接于5之后**
  - 后续冲突元素依次**尾插**入链表
  - 链表末尾指针域置空

1.例题:链地址法应用 03:02:53

【题目条件】

- 散列函数：**H(key) = key mod 13**
- 表长：**14**（取略小于除数13的质数，地址范围0~13）
- 冲突处理：**链地址法**

【散列地址计算与链表构建】

- 26 mod 13 = 0 → 挂接于地址0
- 41 mod 13 = 2 → 挂接于地址2
- 25 mod 13 = 12 → 挂接于地址12
- 05 mod 13 = 5 → 挂接于地址5
- 07 mod 13 = 7 → 挂接于地址7
- 15 mod 13 = 2 → 冲突 → **接于41之后**
- 12 mod 13 = 12 → 冲突 → **接于25之后**
- 49 mod 13 = 10 → 挂接于地址10
- 51 mod 13 = 12 → 冲突 → **接于12之后**
- 31 mod 13 = 5 → 冲突 → **接于05之后**

- 62 mod 13 = 10 → 冲突 → **接于49之后**

【链表收尾】

- 各链表**末节点指针域置空**